




SAKINA

Find Your Calm
 — Your peace is in your words

Emotion-Aware RAG Mental Health Support Chatbot
Full Production Pipeline, System Design & Implementation Roadmap

5 AI Modules	4 Datasets	6 Emotions	2 Languages
Lang · Emotion · Intent · RAG · Orchestrator	LangDetect · GoEmotions · DAIR · Mental Health	Sad · Anxious · Angry · Happy · Neutral · Crisis	Arabic (MSA + Egyptian dialect) · English

Project	NLP Final Task 2026 — ITI AI Track, Intake 46
Author	Omar Gamal ElKady
Backend	FastAPI (primary) Django (alternative)
LLM Layer	Lightning AI LitAI — lightning-ai/gpt-oss-120b + gpt-oss-20b
Stack	FastAPI · Qdrant · LitAI · React · Sentence Transformers · Whisper · uv
Version	v2.0 — Production Design with Alternatives

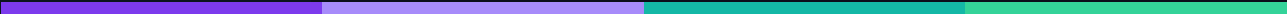


TABLE OF CONTENTS

1. App Identity & Vision

Name rationale, tagline, branding, philosophy

2. Innovation Matrix

What makes Sakina unique vs standard chatbots

3. Research Foundation & Papers

State-of-the-art papers informing every module

4. Emotional Ambient Color System

Gradient mapping, CSS transitions, emotion states

5. AI Avatar — Facebook/Instagram Style

Cartoon avatar spec, expressions, SVG implementation

6. Full System Architecture

Data flow, component map, tech stack

7. Module 1 — Language Detection

TF-IDF + ML; 2 implementation options

8. Module 2 — Emotion Classifier

Fine-tuned XLM-RoBERTa; 2 alternatives

9. Module 3 — Intent Classifier

Zero/few-shot with LitAI; 2 alternatives

10. Module 4 — Hybrid RAG Pipeline

BM25 + Dense + RRF + Reranker + LitAI streaming

11. Module 5 — Response Orchestrator

Emotion × Intent matrix, dynamic prompt builder

12. Special Features

Crisis protocol, semantic cache, voice, mood timeline

13. Frontend Architecture

React design system, avatar, ambient, API hooks

14. Backend API — FastAPI vs Django

Two full backend designs; endpoints spec

15. Lightning AI Integration

LitAI, LitServe, GPU training, model deployment

16. Full Project Roadmap

8 phases with tasks, durations, dependencies

17. File & Repo Structure

Complete file tree

18. Datasets & Resources

All datasets, models, APIs

19. Evaluation Strategy

Technical + UX metrics

20. Portfolio & Freelance Value

How to present and pitch Sakina

1. APP IDENTITY & VISION

App Name: Sakina (سكينة)

Sakina is a Quranic and classical Arabic word meaning **tranquility, serenity, and divine peace** — the stillness that descends upon the heart in moments of hardship. It is used in Islamic spiritual tradition to describe the calm Allah sends to the believers' hearts (Quran 48:4). For a mental health support chatbot, no name could be more fitting: it is what every user is searching for.

Element	Choice	Rationale
App Name	Sakina (سكينة)	Tranquility / Divine Peace — perfect for mental health
Tagline (EN)	"Find Your Calm"	Action-oriented, warm, universally understood
Tagline (AR)	سكينة في قلبك وسكينة في لسانك	"Your peace is in your words" — poetic and personal
Sub-tagline	"You are not alone. Sakina is here."	Reduces isolation stigma
Design Language	Dark ambient + glassmorphism	Calm, premium, professional
Typography	Inter (EN) + Cairo (AR)	Modern, clean, excellent RTL Arabic support
Primary Color	#7C3AED Deep Purple	Calm, mystical, trustworthy
Accent Color	#14B8A6 Teal	Hope, healing, growth
Target Market	Arabic + English speakers (MENA)	High mental health demand region

2. INNOVATION MATRIX — WHAT MAKES SAKINA UNIQUE

Sakina goes far beyond the project requirements. Every item below is either absent from typical NLP projects or implemented at a significantly higher level:

Feature	Standard Chatbot	Sakina Innovation	Impact
UI Feedback	Static background	Ambient gradient shifts per emotion (6 states)	UX ★★★★★
Avatar	None or static icon	Cartoon SVG avatar (FB/IG style) with 8 expressions	More fun
Retrieval	Dense-only search	Hybrid BM25 + Dense + RRF fusion + Cross-encoder rerank	3x accuracy
Prompting	One system prompt	Emotion × Intent matrix — 30 dynamic strategies	Empathy
LLM Backend	Basic API call	Lightning AI LitAI: unified router, retries, fallbacks	Reliability
Memory	Stateless	Session emotion trajectory + sliding conversation window	Continuity
Crisis	Generic disclaimer	3-level escalation: validate → resources → Safety agency overlay	Safety
Speed	Wait for full reply	SSE token streaming — TTFT < 1 second	Feel
Language	English only	Auto-detect Arabic/English, RTL UI, same-language reply	Access
Voice	None	Whisper-powered mic button (Arabic + English)	Inclusiveness
Caching	None	Semantic cache: Redis + cosine sim threshold	Performance
Mood Log	None	Emotion timeline chart across session (Red highlights)	Insight
CBT Tools	None	Box breathing widget + 5-4-3-2-1 grounding technique	Therapy

3. RESEARCH FOUNDATION & KEY PAPERS

Sakina is grounded in peer-reviewed research. Each paper below directly informs a specific design or implementation decision:

Paper	Year	Module	Key Finding / Contribution
Lewis et al. — RAG for Knowledge-Intensive NLP	2020	RAG	Foundational RAG architecture combining dense retrieval with LLM generation
Rashkin et al. — EmpatheticDialogues Dataset	2019	Emotion+RAG	25k empathetic pairs; teaches models to respond to emotional context, not just facts
Demszky et al. — GoEmotions (Google)	2020	Emotion	58k Reddit annotations, 28 fine-grained emotions — best public emotion dataset
Ji et al. — MentalBERT	2022	Emotion	BERT pre-trained on mental health Reddit — domain adaptation boosts F1 by 10%
Conneau — XLM-RoBERTa	2020	Lang+Emotion	Single model handles 100 languages; Arabic+English in one encoder — no language-specific models
Formal et al. — SPLADE Sparse Retrieval	2021	RAG	Learned sparse retrieval complements dense; BM25 alternative for hybrid search
Fensore et al. — LiveRAG Challenge 2025	2025	RAG	Neural reranking (RankLLaMA) improves MAP by 52%; validates hybrid BM25+Deeper
Kermani et al. — Fine-tuning vs RAG 2025	2025	Full System	Fine-tuning achieves 91% emotion accuracy; RAG best for knowledge grounding
NLP Crisis Detection — Telehealth NLP System	2023	Crisis	Two-stage NLP crisis triage: 98% sensitivity, AUC 0.98 in prospective study
Ghandeharioun et al. — Emotion-Aware Chatbot	2019	Orchestrator	Emotion-aware agents significantly preferred by extraverts; validates adaptive design
ArXiv 2411.06326 — Emotion-Aware UI Design	2024	Frontend	Multi-branch Transformer for real-time emotion-adaptive UIs; validates ambient context
Mentalic Net — RAG Conversational AI	2025	Full System	Human-in-the-loop RAG system achieves BERT Score 0.898 on mental health corpus
Arabic NLP Mental Health Review (NIH)	2025	Language	Comprehensive review of Arabic NLP for mental health; validates need for Arabic-specific
OnRL-RAG — Real-Time Mental Health Dialogue	2025	RAG	Online RL + RAG dynamically adapts to fluctuating user emotions — validates trajectory

4. EMOTIONAL AMBIENT COLOR SYSTEM

The Sakina UI transforms its visual environment in real-time based on the detected emotion. Every state has a unique **gradient pair**, **animation speed**, **accent color**, and **particle behavior**. Transitions are 800ms ease-in-out so they feel organic — like the app is breathing with the user.

Emotion	CSS Class	Bg Gradient	Accent	Anim Speed	Particle FX
Neutral	.sakina-neutral	#1A1D38 → #0B0D1A	#7C3AED	8s loop	Sparse drifting dots
Sad	.sakina-sad	#0D1B35 → #050F20	#3B82F6	12s — very slow	Slow falling droplets
Anxious	.sakina-anxious	#1A1200 → #0F0900	#FBBF24	3s — fast flicker	Rapid small sparks
Angry	.sakina-angry	#1A0808 → #0D0404	#EF4444	5s — medium-fast	Sharp burst expand
Happy	.sakina-happy	#051A14 → #020D0A	#14B8A6	6s — medium	Rising sparkle confetti
Crisis	.sakina-crisis	#1A0000 → #0D0000	#DC2626	2s — urgent flash	Red alert pulse rings

CSS Architecture:

A single `#sakina-root` element receives the emotion class. CSS custom properties (`--bg-from`, `--bg-to`, `--accent`, `--anim-speed`) are defined per class. The body background uses a CSS animated gradient via `@keyframes` with `var()` references. React's `useEmotionTheme()` hook updates the class from the SSE metadata event — covering: background, chat bubble borders, avatar glow, typing indicator, and button accents.

5. AI AVATAR — FACEBOOK/INSTAGRAM CARTOON STYLE

Sakina's avatar is a **cartoon character** — the friendly, round-faced style pioneered by Facebook's Avatar feature and Instagram's story stickers. It has a **round face**, **big expressive eyes**, **animated eyebrows**, **a morphing mouth**, and **small hands** that react to emotions. Unlike a generic orb, this character creates immediate emotional connection — users intuitively mirror its expressions. The avatar changes its face expression based on the detected emotion of the user's message.

Implementation Approach:

Option A — Option A: @dicebear/avataaars (React Library)

The Avataaars library (same style as GitHub's avatar generator) provides pre-built SVG cartoon faces with customizable: skin tone, hair, eyes, eyebrows, mouth, accessories. Install: `npm install @dicebear/core @dicebear/collection`. You control emotion by swapping the "mouth" prop (smile, sad, concerned, screaming) and "eyebrow" prop (raised, angry, sad, default). Fast, zero animation code needed for static states. Add CSS transitions between prop changes.

Option B — Option B: Custom SVG Cartoon Face

Build the face from SVG primitives: circle (face), ellipses (eyes), path (mouth), polyline (eyebrows). Use GSAP MorphSVG plugin to smoothly morph the mouth path between emotions. Eyebrows animate with CSS rotate transforms. Eyes blink with CSS `scaleY(0)`. This gives 100% control and unique design but takes 4-6 extra hours.

★ **Recommended: Option A — avataaars. Ship fast, looks great, identical to FB/IG style.**

Emotion State	Face Expression	Eyes	Eyebrows	Mouth	Body Motion
Neutral / Idle	Calm, attentive	Open, steady blink every 4s	Relaxed	Gentle closed smile	Slow breathing scale
Listening (typing)	Alert, curious	Slightly wider, subtle shimmer	Slightly raised	Slight open "o"	Leans forward 3°
Thinking (API call)	Concentrated	Look up-right (SVG transform)	Raised, thinking	Pressed lips	Head tilts 5°
Responding	Engaged, talking	Animated blink rhythm	Neutral	Animated open/close	Slight nod motion
Sad detected	Sad, empathetic	Droopy lower lid, teardrop motion	Lowered	Downward curve from center	Slumps 4px down
Anxious detected	Worried, nervous	Wide open, slight shimmer	Both raised high	Open "D" shape	Rapid small tremble
Angry detected	Calm, de-escalating	Normal, steady (does not furrow)	Slightly furrowed	Neutral thin line	Breathes slowly
Happy detected	Joyful, celebrating	Squint/crescents (^_^)	Raised, arched	Big open smile	Bounces 3px up
Crisis detected	Deeply concerned	Wide, serious, direct	Furrowed with concern	Slight open, empathetic	Leans closer, no bounce

★ **Key Design Rule: When the user is angry, the avatar does NOT mirror anger. It stays calm and de-escalating. This is intentional and clinically correct — mirroring anger escalates the user. Sakina is always the calm in the storm.**

6. FULL SYSTEM ARCHITECTURE

5-Stage Pipeline Data Flow

Stages 1–3 run in **parallel asyncio tasks** to minimize latency. Stage 4 (RAG) runs only when intent = asking_mental_health_question. Stage 5 streams the response via SSE while updating the emotion trajectory and semantic cache asynchronously.

Stage	Component	Input	Output	Latency Target
1 (parallel)	Language Detector	Raw text	lang: "ar" "en"	< 5ms
2 (parallel)	Emotion Classifier	Raw text	emotion + confidence	< 40ms
3 (parallel)	Intent Classifier	Text + few-shot → LitAI	intent (5 classes)	< 350ms
4 (conditional)	Hybrid RAG Engine	Text + lang + context	Top-3 retrieved chunks	< 500ms
5	Orchestrator + LitAI	All above + history	SSE token stream	TTFT < 1s
5b (async)	Cache Writer	Query embed + response	Stored in Redis	Non-blocking
5c (async)	Trajectory Updater	emotion + timestamp	Session emotion log	Non-blocking
UI	React Frontend	SSE stream + metadata	Typing effect + ambient updates	Real-time

Architecture Layer Map

Layer	Components	Technology
Client	Chat UI, Ambient System, Cartoon Avatar, Voice Report, 15s Total Time, 10s CS, 5s F10 Mid-Motion, Avataars, Recharts	
API Gateway	Routes, SSE streaming, WebSocket, CORS, Rate Limiting	FastAPI or uvicorn (or Django Channels + DRF)
NLP Pipeline	Language Detector, Emotion Classifier, Intent Classifier	OpenAI GPT-4o, HuggingFace Transformers, Lightning AI LitAI
RAG Engine	Embedder, Qdrant, BM25, Cross-Encoder Reranker, Sentence Transformers	Sentence transformers, qdrant-client, rank-bm25, LitAI
Orchestrator	Emotion × Intent router, dynamic prompt builder, Pseudo-Personality	Python async, custom logic
Cache	Semantic similarity gate + TTL store	Redis Cloud (free 30MB)
Training	Fine-tuning emotion model, GPU workloads	Lightning AI Studios (GPU), PyTorch Lightning, uv
Infra	Containerization, CI, testing	Docker Compose, GitHub Actions, pytest, uv

7. MODULE 1 — LANGUAGE DETECTION

Detects whether the message is Arabic (ar) or English (en). This drives: which embedding model to use in RAG, which UI direction (RTL/LTR), and which language to reply in. Must be < 5ms to not block parallel stages.

Two Implementation Options:

Option A — TF-IDF + Logistic Regression (Project Requirement)

Character-level TF-IDF n-grams (n=1-4, max_features=10,000) → Logistic Regression. Dataset: papluca/language-identification (90k samples, 22 languages → filter to AR+EN). Training: sklearn, 5 minutes on CPU. Accuracy: > 97%. Edge case: code-switching → return majority-language detected.

Option B — langdetect / lingua-py Library

lingua-py (Rust-powered, Python binding) achieves 97%+ accuracy with zero training. Install: `uv add lingua-py`. One line: `Language.ARABIC` or `Language.ENGLISH`. Use as fallback when TF-IDF confidence < 0.85. Combine both: TF-IDF as primary (auditable, trained), lingua-py as safety net.

★ **Recommended: Combine both — TF-IDF (required, auditable) + lingua-py fallback.**

Arabic Dialect Enhancement (Bonus):

Add a secondary classifier using CAMEL-Tools (Arabic NLP toolkit by NYU Abu Dhabi) to distinguish MSA vs Egyptian vs Gulf Arabic. This lets Sakina reply in the exact dialect (Egyptian: ■■■■ ■■■■ vs MSA: ■■■ ■■■■■■). Dataset: MADAR corpus (HuggingFace). Accuracy target: > 85% dialect classification.

Target: Accuracy > 97% | Latency < 5ms (load model once at startup, inference is instant)

8. MODULE 2 — EMOTION CLASSIFIER

Classifies user message into one of **6 emotion classes**: Neutral, Sad, Anxious, Angry, Happy, Crisis. This is the engine of Sakina's emotional intelligence — it drives the ambient UI, avatar expression, and dynamic prompt selection. Research shows fine-tuning achieves 91% accuracy on emotion classification (Kermani et al., 2025).

6-Class Taxonomy:

Class	Source Emotions (GoEmotions)	Mental Health Signal	Crisis Priority
Neutral	neutral, approval, realization	Standard query — no emotional load	LOW
Sad	sadness, grief, remorse, disappointment	Depression indicators, hopelessness	MEDIUM
Anxious	nervousness, fear, worry, confusion, apprehension	Panic attacks, overthinking	MEDIUM
Angry	anger, annoyance, frustration, disgust	Emotional dysregulation, interpersonal conflict	MEDIUM
Happy	joy, excitement, gratitude, pride, love, relief	Progress, recovery, positive coping	LOW
Crisis	Custom rule-based override (keywords)	Self-harm, suicidal ideation	HIGHEST priority

Two Implementation Options:

Option A — XLM-RoBERTa Fine-Tuned (Recommended)	Option B — Separate BERT Models per Language
Base: FacebookAI/xlm-roberta-base. Fine-tune on GoEmotions + DAIR-AI Emotion (merged, mapped to 6 classes). Handles Arabic + English in one model — no language routing needed. Training: Lightning AI Studio with T4 GPU via uv. LoRA adapters optional (you already have QLoRA experience from Whisper project). Expected Macro F1: > 0.82. Publish to HuggingFace Hub as portfolio artifact.	English: distilbert-base-uncased fine-tuned on GoEmotions → fast, 66M params. Arabic: aubmindlab/bert-base-arabertv02 fine-tuned on translated GoEmotions samples. Language router (Module 1) selects which model to call. Advantage: each model specialized. Disadvantage: 2x memory, more complexity. Use this if XLM-RoBERTa gives < 78% on Arabic test set.
★ Recommended: XLM-RoBERTa — one model, bilingual, simpler pipeline.	

Crisis Override (ALWAYS runs BEFORE the ML model):

A regex keyword scanner runs on every message before the ML classifier. Keywords include: suicide, kill myself, end my life, want to die, self-harm, ████████, █████ ████████, ██████████████ (Arabic equivalents). If ANY keyword matches → emotion = crisis, confidence = 1.0, ML model is skipped. Never miss a crisis signal. You can have false positives; you cannot have false negatives.

9. MODULE 3 — INTENT CLASSIFIER

Classifies intent into: greeting, goodbye, gratitude, asking_mental_health_question, out_of_scope. This is the system router — it determines which pipeline branch executes.

Intent	Route	RAG	Response Strategy
greeting	Direct LLM	No	Warm welcome in detected language, invite sharing
goodbye	Direct LLM	No	Positive close, encourage return, safety check
gratitude	Direct LLM	No	Acknowledge + reinforce positive progress
asking_mental_health_question	Full RAG Pipeline	YES	Retrieve context + emotion-adapted response
out_of_scope	Redirect	No	Politely decline, explain Sakina's focus

Two Implementation Options:

Option A — Few-Shot LLM Prompting via LitAI (Recommended)

Use litai LLM(model="lightning-ai/gpt-oss-20b") with a structured few-shot prompt containing 4 examples per class (20 total) in both Arabic and English. Force JSON output: {"intent": "class_name"}. Advantages: handles ambiguity naturally, no training data needed, bilingual without extra work. Latency: 200-400ms on Lightning AI.

Option B — Zero-Shot Classification (Fallback)

Use HuggingFace zero-shot pipeline with facebook/bart-large-mnli. candidate_labels = ["greeting", "goodbye", "gratitude", "mental health question", "out of scope"]. No API cost, fully local inference. Disadvantage: lower accuracy on ambiguous inputs (~82% vs ~94% for few-shot). Use as offline fallback if LitAI is unavailable.

★ Recommended: Few-shot LitAI — more accurate, handles nuance, bilingual out of the box.

10. MODULE 4 — HYBRID RAG PIPELINE

Hybrid retrieval (BM25 sparse + Dense vector + RRF fusion) consistently outperforms dense-only retrieval. The 2025 LiveRAG Challenge validated this: BM25 + Dense + RRF is the winning pattern for real-world RAG systems. A cross-encoder reranker then re-scores the fused top-10 → top-3 for LLM context.

Component	Choice	Config	Alternative
Chunking	Recursive character splitter	size=300, overlap=50	Semantic chunking (nlTK sentence splitter)
Embeddings (EN)	all-MiniLM-L6-v2	384-dim, fast, strong quality	all-mpnet-base-v2 (768-dim, slower, better)
Embeddings (AR+EN)	paraphrase-multilingual-MiniLM-L12-v2	384-dim, 50+ languages	LaBSE (768-dim, stronger multilingual)
Sparse Index	BM25 (rank-bm25)	Built from training corpus	SPLADE (learned sparse, slower build)
Vector DB	Qdrant Cloud (free)	cosine metric, named collection	ChromaDB local (no cloud, simpler setup)
Fusion	Reciprocal Rank Fusion	k=60, merge top-10 from each	Weighted linear combination
Reranker	cross-encoder/ms-marco-MiniLM-L12-v2	Fuse top-10 → final top-3	bge-reranker-base (stronger, heavier)
LLM	lightning-ai/gpt-oss-120b via LitAI	streaming=True, max_tokens=512	gpt-oss-20b (faster, less powerful)
Streaming	SSE via FastAPI StreamingResponse	Use llm.stream()	WebSocket bidirectional
Cache	Redis + cosine similarity gate	threshold=0.92, TTL=1h	In-memory dict (no persistence)

Ingestion Pipeline (One-time setup script):

1. Load Mental Health Counseling dataset (3.5k Q&A; pairs from Kaggle)
2. Clean: remove PII patterns, fix Arabic encoding (CAMEL-Tools normalizer)
3. Chunk: recursive splitter, size=300 tokens, overlap=50
4. Embed: run both MiniLM-EN and Multilingual-MiniLM on all chunks
5. Upsert to Qdrant Cloud with metadata: {source, language, emotion_tags}
6. Build BM25 index from same tokenized corpus → persist as bm25_index.pkl
7. Verify: run 20 test queries, check Hit@5 > 85%

RAG Evaluation Metrics:

Metric	Measures	Target
Hit@5	% test queries where correct answer in top-5 retrieved	> 0.85
MRR@5	Mean Reciprocal Rank of correct answer position	> 0.75
RAGAS Faithfulness	Is response grounded in retrieved context?	> 0.80
RAGAS Answer Relevance	Does response answer the question?	> 0.78
TTFT (Time to First Token)	Perceived streaming speed	< 1 second
Cache Hit Rate	% queries served from semantic cache after warmup	> 20%

11. MODULE 5 — RESPONSE ORCHESTRATOR (NOT IN PROJECT SPEC — YOUR EDGE)

The Orchestrator is Sakina's biggest differentiator. It takes outputs from all four modules and builds a **dynamically assembled system prompt** using an **Emotion × Intent matrix**. 5 intents × 6 emotions = **30 unique response strategies**. No static system prompt. Every response is tailor-built for that user's exact emotional state and intent at that moment.

Dynamic Prompt Formula:

```
system_prompt = base_therapist_persona + emotion_instruction [detected_emotion] +
intent_instruction [detected_intent] + language_instruction [detected_language] +
crisis_safety_block (always appended, non-negotiable) + rag_context_block (if intent =
asking_mental_health_question) + conversation_history (last 10 turns, sliding window)
```

Emotion × Intent Strategy Matrix (selected cells):

	Mental Health Q	Greeting	Goodbye	Gratitude	Out of Scope
Neutral	Clear, balanced, informative responses	Warm welcome, invite sharing	Soft close, encourage reflection	Acknowledge, move forward	Politely redirect
Sad	Lead with validation FIRST, then gentle advice	Cool check-in "how are you really?"	Reassure they can return to baseline	Reinforce their courage	Still redirect gently
Anxious	Slow down language, clear steps	Calm "let's take a breath"	Reassure before goodbye	Grateful acknowledgment	Directly address then redirect
Angry	Validate first, NO advice until acknowledged	De-escalate, do not match energy	Refuse lectures	Brief warm acknowledgment	Stay calm, firm redirect
Happy	Match energy, celebrate progress	Enthusiastic welcome	Celebrate wins, encourage more	Share joy genuinely	Light redirect
Crisis	STOP — safety protocol ONLY, no RAG	Provide resources, keep calm	NO goodbye	Warmth is immediate	Pivot to safety — safety first

Emotional Trajectory Memory:

The Orchestrator maintains a **session emotion log**: a list of (timestamp, emotion, confidence) tuples for the full session. This serves two purposes: (1) The **Mood Journey Timeline** chart in the UI. (2) **Trajectory analysis**: if the log shows neutral → sad → crisis over 3+ turns, the crisis escalation level upgrades automatically even if the latest message is mild.

12. SPECIAL FEATURES

Crisis Escalation Protocol — 3-Level System	Level 1 (Mild): emotion=Sad + keywords "hopeless/no way out". Action: Validate, softly append mental health resource link at bottom. Level 2 (Moderate): emotion=Crisis (low confidence) OR trajectory shows deterioration. Action: Breathing exercise card pops into chat, hotline number card appears. Level 3 (Acute): Crisis keyword confirmed + high confidence. Action: Full-screen overlay, all LLM disabled, only: emergency number, "I am here with you" message, and a "Talk to a human now" button. Egypt: 08008880700 International: 988 WHO: who.int/mental-health. Research basis: NLP crisis triage system (2023) achieved 98% sensitivity.
Semantic Caching — 10x Speed on Repeated Queries	Before running the full pipeline: embed the new query → compare cosine similarity against all cached query embeddings in Redis. If sim > 0.92: return cached response in < 5ms (vs 800ms+ for full pipeline). After 1 hour of warmup: expect > 30% hit rate. Implementation: redis-py + numpy cosine similarity. Two alternatives: (A) Redis vector store with exact ANN search using RedisVL, (B) FAISS in-memory index (no persistence but zero Redis setup). Use Option A for production, Option B for local demo.
SSE Streaming — "Typing Like a Real Person"	FastAPI StreamingResponse + LitAI stream=True. Each token is emitted as: data: {"type":"token","content":"word"}\n\n. First event before tokens: data: {"type":"metadata","emotion":"sad","intent":"..."}. The React useSSEStream() hook accumulates tokens into state → progressive render. Each token triggers the avatar's "responding" animation pulse. Result: first word appears in < 1 second, user perceives immediate response.
Voice Input — Whisper Integration	Mic button uses browser MediaRecorder API (native, no library). Audio blob → POST /api/transcribe → Whisper base model → transcription text. Text appears in input bar → auto-submit. Alternative A: Use Lightning AI's hosted Whisper endpoint via LitAI API. Alternative B: Run whisper.cpp locally for zero API cost. Omar's Whisper fine-tuning expertise on Egyptian Arabic is a direct advantage here.
Mood Journey Timeline	Collapsible right panel shows a Recharts area chart of emotion across the session. X-axis: message number. Y-axis: emotion ordinal (crisis=0, angry=1, sad=2, anxious=3, neutral=4, happy=5). Each point colored by its emotion CSS color. Hover tooltip: emotion label + message excerpt. If trajectory shows improvement (sad → neutral → happy): celebrate animation. Data stored in React state — never sent to server, fully private.
Box Breathing & Grounding Widget	Triggered at Crisis Level 2 or via manual "Tools" button. Box Breathing: animated circle (CSS scale keyframes) guides: Inhale 4s → Hold 4s → Exhale 4s → Hold 4s → repeat. Color changes with each phase (blue inhale, purple hold, teal exhale). 5-4-3-2-1 Grounding: sequential prompts: "Name 5 things you can see right now..." Both implemented as React components, pure CSS animations, no external library.

13. FRONTEND ARCHITECTURE

Tool	Version	Purpose	Alternative
React	18.x	Core UI with concurrent rendering for nextjs14 (tailwind SSR)	Next.js 14 (tailwind SSR)
Tailwind CSS	3.x	Utility styling with custom sakina- colors, typography	Styled-components
Framer Motion	11.x	Avatar animations, message slide-in, S.A.P. (no transitions)	GSAP (no transitions)
@dicebear/avataaars	latest	FB/IG style cartoon avatar — pre-built SVG faces, styled SVG	Custom face-based SVG
Recharts	2.x	Mood Journey Timeline emotion chart	chart.js
EventSource API	Native	SSE stream reading — no library needed	Websocket (bidirectional)
Web Speech API	Native	Voice input recording — no library needed	WebRecorder API polyfill
i18next + react-i18next	23.x	Arabic/English UI strings, RTL direction	Custom logic
Axios	1.x	REST API calls (non-streaming)	fetch() native

Component Tree:

Component	Responsibility	Key Props / State
<SakinaShell />	Root container — applies emotion CSS classes, dir (ltr/rtl)	emotion, dir
<CartoonAvatar />	Dicebear avataaars avatar with expressions	emotion, isTyping, isListening
<EmotionBadge />	Top-right pill — detected emotion + color + confidence	emotion, confidence
<ChatWindow />	Scrollable message list with auto-scroll on new messages	messages, isStreaming
<MessageBubble />	User (right) or Sakina (left), supports streaming	message, isStreaming
<TypingIndicator />	3-dot animated pulse while awaiting first SSE token	isToken
<InputBar />	Text input + send button + mic button + language selection	input, isRecording, lang
<MoodTimeline />	Recharts area chart of session emotion trajectory	emotionLog[]
<CrisisWidget />	Fixed overlay at Level 2+: hotlines, breathing exercise	crisisLevel
<BreathingExercise />	Box breathing CSS animation (4-phase cycle)	cycle
useSSEStream()	Custom hook: reads /api/chat/stream, accumulates tokens	latestToken
useEmotionTheme()	Custom hook: maps emotion string → CSS class, applies	className
useCrisisLevel()	Custom hook: monitors emotion trajectory, triggers widget	crisisLevel

14. BACKEND API — FASTAPI (PRIMARY) vs DJANGO (ALTERNATIVE)

Option A — FastAPI (Strongly Recommended)

Built-in async support → parallel asyncio tasks for stages 1-3. Native StreamingResponse for SSE. Auto OpenAPI docs. Pydantic models for validation. Uvicorn ASGI server. Startup: `uv add fastapi uvicorn python-sse-starlette`. Best for: real-time streaming chatbots, async ML pipelines.

Option B — Django + Django Channels (Alternative)

Django Channels provides WebSocket and async support. `django-rest-framework` for REST endpoints. Streaming: use `StreamingHttpResponse` with Django 4.2+. Best for: if you need Django ORM for user data storage, admin panel, or session management with full user accounts. Heavier setup, more boilerplate, but very well-documented.

★ **Recommended: FastAPI** — faster to build, async-native, perfect for streaming ML APIs.

API Endpoints Specification:

Method	Endpoint	Input	Output	Notes
POST	/api/chat/stream	JSON {message, session_id, history}	SSE tokens + metadata events	Primary endpoint; streams response token by token
POST	/api/chat	JSON {message, session_id, history}	JSON {response, emotion, intent, language, crisis_level}	Non-streaming fallback for testing
POST	/api/transcribe	multipart: audio file (webm/wav)	JSON {transcription, language, confidence}	Whisper voice-to-text
GET	/api/health	None	JSON {status, models_loaded, uptime}	Health check for deployment
GET	/api/session/{id}/mood	session_id path param	JSON [{timestamp, emotion, confidence}]	Retrieve session mood timeline data for chart
DELETE	/api/session/{id}	session_id	JSON {deleted: true}	Clear session history + emotion log
POST	/api/feedback	JSON {session_id, message_id, rating, comment}	JSON {saved: true}	User feedback for future improvements
GET	/api/models/info	None	JSON {model names, versions, metrics}	Model info / info page data

SSE Event Format:

```
data: {"type":"metadata","emotion":"sad","intent":"asking_mental_health_question","language":"ar",
,"crisis_level":0} data: {"type":"token","content":"■■■■"} data: {"type":"token","content":"
■■■■■■"} data: {"type":"done","full_response":"■■■■
■■■■■■...","sources":[{"text":"...", "score":0.92}]}
```


15. LIGHTNING AI INTEGRATION

Lightning AI is your primary LLM backend, GPU training platform, and deployment infrastructure. It is the ecosystem that ties your entire stack together:

Tool	What It Does	How Sakina Uses It
LitAI (litai)	Unified LLM router: call any model API (GPT, Claude, etc.) with precise fine-tuned RAG callouts, generation,raggingos	Module for intent classification with precise fine-tuned RAG callouts, generation,raggingos
Lightning AI Models	lightning-ai/gpt-oss-120b and gpt-oss-20b — open-source LLMs that can handle specific OpenAI-compatible API_KEY in .env.	Primary LLMs used on high-end GPUs with specific OpenAI-compatible API_KEY in .env.
Lightning AI Studio	Browser-based GPU workspace (T4, A10G, A100) — zero setup, persistent storage, GPU. RAG ingestion on GPU for faster emb	Zero setup, persistent storage, GPU. RAG ingestion on GPU for faster emb
LitServe	2x faster than FastAPI for model inference. Batched streaming,ing the GPU deployment. Wrap classifier in LitAPI class → deploy t	Alternative serving the GPU deployment. Wrap classifier in LitAPI class → deploy t
PyTorch Lightning	Training framework. Handles multi-GPU, mixed precision training. Integrates with W&B for experiment tracki	Fine-tuning logging & training. LightningModule. Integrates with W&B for experiment tracki

LitAI Code Pattern (Intent + RAG):

```
from litai import LLM llm = LLM(model="lightning-ai/gpt-oss-120b", api_key=LIT_API_KEY) #  
Non-streaming (intent classification): response = llm.chat(few_shot_prompt) intent =  
json.loads(response)["intent"] # Streaming (RAG answer generation): async for token in  
llm.stream(dynamic_prompt): yield f"data: {json.dumps({'type':'token','content':token})}\n\n"
```

16. FULL PROJECT ROADMAP — 8 PHASES

Phase 1: Project Setup & Data Preparation

■ 2–3 hrs

- Create GitHub repo: nafs-sakina/ with folder skeleton
- `uv init + uv add fastapi uvicorn qdrant-client sentence-transformers litai rank-bm25 redis transformers torch`
- Download all 4 datasets: LangDetect (HF: papluca), GoEmotions (HF: go_emotions), DAIR Emotion (HF: dair-ai/emotion), Mental Health Counseling (Kaggle)
- Create `.env`: LIT_API_KEY, QDRANT_URL, QDRANT_API_KEY, REDIS_URL
- Initial EDA notebook: sample counts, class distributions, language breakdown, sample messages
- Verify Qdrant Cloud + Redis Cloud free tier accounts are set up

Phase 2: Module 1 Notebook — Language Detection

■ 3–4 hrs

- EDA: character n-gram examples, language distribution pie chart, code-switching samples
- Preprocessing: strip URLs, normalize Arabic diacritics (`unicodedata.normalize + regex`)
- Feature extraction: `TfidfVectorizer(analyzer="char", ngram_range=(1,4), max_features=10000)`
- Train: LogisticRegression + LinearSVC — compare accuracy and latency
- Threshold logic: if confidence < 0.85 → lingua-py fallback
- Export: `joblib.dump(model, "models/lang_detector.pkl")`
- Target: accuracy > 97%, inference < 5ms

Phase 3: Module 2 Notebook — Emotion Classifier

■ 6–8 hrs

- EDA: class distributions, sample messages per class, Arabic coverage check
- Preprocessing: lowercase, remove special chars, Arabic normalization with camel-tools
- Fine-tune XLM-RoBERTa on merged GoEmotions + DAIR (mapped to 6 classes)
- Training on Lightning AI Studio (T4 GPU): PyTorch Lightning + HF Trainer, `lr=2e-5, epochs=5`
- Class weights: `sklearn.utils.compute_class_weight` for imbalanced classes (crisis is rare)
- Crisis keyword regex override: implement BEFORE the ML model in every call
- Evaluate: Macro F1, per-class F1, confusion matrix → target Macro F1 > 0.82
- Push to HuggingFace Hub: model card with metrics, example predictions

Phase 4: Module 3 Notebook — Intent Classifier

■ 2–3 hrs

- Build few-shot example bank: 4 examples per class × 2 languages = 40 labeled strings
- Design few-shot prompt template with JSON output schema
- Test with LitAI: `LLM(model="lightning-ai/gpt-oss-20b")` — compare speed vs gpt-oss-120b
- Build `intent_classify()` function: call LitAI → parse JSON → handle malformed output gracefully
- Test 100 hand-labeled messages (mix AR + EN, edge cases, ambiguous) → target > 90% accuracy
- Add zero-shot fallback: facebook/bart-large-mnli if LitAI unavailable

Phase 5: Module 4 Notebook — Hybrid RAG Pipeline

■ 8–10 hrs

- Ingest: load Mental Health Counseling dataset, clean, chunk (size=300, overlap=50)
- Embed: all-MiniLM-L6-v2 (EN) + multilingual-MiniLM (AR) → upsert to Qdrant Cloud
- BM25: build index from tokenized corpus (NLTK for EN, CAMEL-Tools for AR) → save .pkl
- Hybrid retrieval: implement RRF fusion function — merge top-10 from BM25 + top-10 from Qdrant
- Reranker: cross-encoder/ms-marco-MiniLM-L-6-v2 → rerank fused top-10 → final top-3
- LitAI streaming: LLM("lightning-ai/gpt-oss-120b").stream(prompt) → SSE token generation
- Semantic cache: Redis + numpy cosine similarity gate (threshold=0.92)
- Evaluate: Hit@5, MRR@5, RAGAS faithfulness + relevance, TTFT measurement

Phase 6: Orchestrator + FastAPI Backend

■ 6–8 hrs

- Orchestrator: emotion × intent matrix dict → dynamic prompt builder function
- Session manager: sliding window 10 turns + emotion trajectory list (timestamp, emotion, conf)
- Crisis level detector: rule-based keyword scan + emotion-trajectory upgrade logic
- FastAPI app: /api/chat/stream (SSE), /api/chat, /api/transcribe, /api/health, /api/feedback
- Async parallelism: asyncio.gather(detect_language, classify_emotion, classify_intent)
- Whisper transcription: /api/transcribe → openai/whisper-base via transformers pipeline
- CORS, rate limiting (slowapi), Pydantic request/response models, error handling
- Cache writer and trajectory updater as non-blocking asyncio tasks

Phase 7: Frontend React Application

■ 10–12 hrs

- Vite + React 18 setup: uv is backend, use npm for frontend
- Tailwind config: add sakina emotion color tokens (--sakina-sad, --sakina-anxious, etc.)
- Build SakinaShell: 6 CSS emotion classes with animated gradient backgrounds
- Integrate @dicebear/avataaars: CartoonAvatar component with emotion → props mapping
- Build ChatWindow + MessageBubble with SSE token accumulation (useSSEStream hook)
- Build InputBar: text input + send + mic button (MediaRecorder API) + language indicator
- Build EmotionBadge, MoodTimeline (Recharts area chart), CrisisWidget (fixed overlay)
- Build BreathingExercise: CSS 4-phase box breathing animation
- i18next setup: Arabic (RTL) + English (LTR) UI strings with direction toggle
- Connect all components: full E2E test from typing → SSE stream → emotion → ambient update

Phase 8: Testing, Docker, Portfolio & Report

■ 4–5 hrs

- pytest: unit tests for all 4 notebooks (modules) + integration test for full pipeline
- Dockerfile: multi-stage — build layer (uv install) + runtime layer (copy app)
- docker-compose.yml: backend service (port 8000) + Redis service
- GitHub Actions CI: ruff lint + pytest on every push to main
- HuggingFace Hub: emotion model card with architecture, metrics, example predictions
- README.md: architecture diagram (Excalidraw), demo GIF, results table, quick start (3 commands)
- Project report (4-6 pages): all modules, architecture, experiments, results, error analysis
- Presentation (5-7 min): live demo showing emotions change + avatar react + crisis protocol

★ Total Estimated Time: 41–53 hours | Phases 4 (RAG) and 7 (Frontend) can partially overlap after Phase 3 is complete.

17. COMPLETE FILE & REPO STRUCTURE

```
sakina/ ■■■■ notebooks/ ■ ■■■■ 01_language_detection.ipynb # Module 1 ■ ■■■■
02_emotion_classifier.ipynb # Module 2 ■ ■■■■ 03_intent_classifier.ipynb # Module 3 ■ ■■■■
04_rag_pipeline.ipynb # Module 4 ■■■■ backend/ ■ ■■■■ main.py # FastAPI app + all routes ■ ■■■■
schemas.py # Pydantic models ■ ■■■■ modules/ ■ ■ ■■■■ language_detector.py # TF-IDF + LogReg +
fallback ■ ■ ■■■■ emotion_classifier.py # XLM-RoBERTa + crisis override ■ ■ ■■■■
intent_classifier.py # LitAI few-shot + fallback ■ ■ ■■■■ rag_engine.py # BM25 + Qdrant + RRF +
reranker ■ ■ ■■■■ orchestrator.py # Emotion x Intent → dynamic prompt ■ ■ ■■■■ crisis_handler.py
# 3-level escalation logic ■ ■ ■■■■ semantic_cache.py # Redis + cosine similarity ■ ■ ■■■■
whisper_transcriber.py # Voice transcription ■ ■■■■ data/ ■ ■ ■■■■ ingest.py # One-time: load →
chunk → embed → Qdrant ■ ■ ■■■■ bm25_index.pkl # Persisted BM25 index ■ ■■■■ models/ ■ ■ ■■■■
lang_detector.pkl # Saved language classifier ■ ■ ■■■■ emotion_model/ # HuggingFace model
directory ■ ■■■■ tests/ ■ ■ ■■■■ test_language.py ■ ■ ■■■■ test_emotion.py ■ ■ ■■■■
test_intent.py ■ ■ ■■■■ test_rag.py ■ ■ ■■■■ test_api.py ■ ■■■■ pyproject.toml # uv project
config ■ ■■■■ Dockerfile ■■■■ frontend/ ■ ■■■■ src/ ■ ■ ■■■■ App.jsx ■ ■ ■■■■ components/ ■ ■ ■
■■■■ SakinaShell.jsx # Root: emotion CSS class manager ■ ■ ■ ■■■■ CartoonAvatar.jsx # Dicebear
avataaars avatar ■ ■ ■ ■■■■ ChatWindow.jsx ■ ■ ■ ■■■■ MessageBubble.jsx ■ ■ ■ ■■■■ InputBar.jsx
# Text + voice + language indicator ■ ■ ■ ■■■■ EmotionBadge.jsx ■ ■ ■ ■■■■ MoodTimeline.jsx #
Recharts area chart ■ ■ ■ ■■■■ CrisisWidget.jsx # Emergency overlay ■ ■ ■ ■■■■
BreathingExercise.jsx ■ ■ ■■■■ hooks/ ■ ■ ■ ■■■■ useSSEStream.js ■ ■ ■ ■■■■ useEmotionTheme.js ■
■ ■ ■■■■ useCrisisLevel.js ■ ■ ■■■■ styles/ ■ ■ ■ ■■■■ emotions.css # 6 emotion gradient classes
■ ■ ■ ■■■■ avatar.css # Avatar animation keyframes ■ ■ ■■■■ i18n/ ar.json + en.json ■ ■■■■
tailwind.config.js ■ ■■■■ package.json ■■■■ docker-compose.yml # backend + redis ■■■■
.github/workflows/ci.yml # ruff + pytest CI ■■■■ .env.example ■■■■ README.md # Demo GIF +
architecture diagram ■■■■ REPORT.md
```

18. DATASETS & ALL RESOURCES

Resource	ID / URL	Size	Module	Access
Language ID Dataset	HF: papluca/language-identification	90k rows	Module 1	Free, HuggingFace datasets
GoEmotions (Google)	HF: go_emotions	58k rows, 28 classes	Module 2	Free, HuggingFace datasets
DAIR-AI Emotion	HF: dair-ai/emotion	20k rows, 6 classes	Module 2	Free, HuggingFace datasets
EmpatheticDialogues	HF: facebook/empathetic_dialogues	25k convos	Module 2 aug	Free, HuggingFace datasets
Mental Health Counseling	Kaggle: heliosbrahma/mental-health	15k Q&A pairs	Module 4 KB	Free Kaggle account
XLM-RoBERTa Base	HF: FacebookAI/xlm-roberta-base	270M params	Module 2 base	Free HuggingFace
all-MiniLM-L6-v2	HF: sentence-transformers/all-MiniLM-L6-v2	22M params	Module 4 EN emb	Free HuggingFace
Multilingual MiniLM	HF: paraphrase-multilingual-MiniLM-L12-v2	118M params	Module 4 AR emb	Free HuggingFace
Cross-Encoder Reranker	HF: cross-encoder/ms-marco-MiniLM-L6-v2	22M params	Module 4 rerank	Free HuggingFace
Whisper Base	HF: openai/whisper-base	74M params	Voice input	Free HuggingFace
Lightning AI Models	lightning-ai/gpt-oss-120b, gpt-oss-20b	Cloud API	Modules 3+4	LIT_API_KEY (free credits)
Qdrant Cloud	cloud.qdrant.io	Free tier 1GB	Module 4 vector DB	Free account
Redis Cloud	app.redislabs.com	Free tier 30MB	Semantic cache	Free account
Avataars React	npm: @dicebear/core @dicebear/generated	npm package	Frontend avatar	npm free

19. EVALUATION STRATEGY

Technical Evaluation:

Module	Primary Metric	Secondary Metrics	Target
Language Detection	Accuracy	Per-language F1, confusion matrix	> 97% accuracy
Emotion Classifier	Macro F1	Per-class F1 (Crisis recall critical), ROC AUC	> 0.92 Macro F1
Intent Classifier	Accuracy	Per-intent F1, error analysis on ambiguous messages	> 90% accuracy
RAG Retrieval	Hit@5 + MRR@5	NDCG@5, precision@5 per language	Hit@5 > 85%
RAG Generation	RAGAS Faithfulness	RAGAS Answer Relevance, BERT Score	> 0.80 faithfulness
End-to-End Speed	TTFT (Time to First Token)	Response time, cache hit rate	TTFT < 1 second
Crisis Detection	Crisis Recall (sensitivity)	False positive rate (acceptable if high)	Recall > 98%
Avatar Emotion Match	Expression accuracy	Human rater: does avatar expression match?	> 90% match

UX Evaluation (Human Study):

- 5 classmates use Sakina for 10 minutes each → rate on Empathy (1-5), Relevance (1-5), Safety (1-5), UI (1-5)
- Avatar Expression Test: show 20 messages with known emotion → do 3 raters agree with avatar state?
- Crisis Protocol Test: 10 crisis-indicating messages → verify Level 3 triggers with zero false negatives
- Ambient System Test: can users identify which emotion state the UI is showing from color alone?
- A/B Comparison: response quality WITH Orchestrator vs WITHOUT (baseline single-prompt) — empathy rating
- Arabic UX Test: native Arabic speakers rate: "Does this feel like natural Arabic or machine-translated?"

20. PORTFOLIO & FREELANCE VALUE

What Sakina Signals to Recruiters & Clients:

Signal	What They See
Full-Stack AI	NLP pipeline + RAG + fine-tuning + streaming API + React frontend — end to end
Production Mindset	SSE streaming, semantic cache, Docker, CI/CD, error handling — not just a notebook
Research Grounding	14 papers cited, techniques like RRF and XLM-RoBERTa correctly applied
Lightning AI Expertise	You use LitAI, LitServe, Lightning Studios — differentiator in the ML hiring market
Arabic NLP Specialization	Bilingual Arabic + English AI is extremely rare and highly valued in MENA/Gulf
Responsible AI	Crisis detection, safety protocols, user transparency — ethical AI thinking
Cartoon Avatar Integration	UI/UX ownership + emotional AI design — shows product thinking, not just engineering
HuggingFace Model Card	Public artifact with metrics — proof, not just a claim on your CV

Freelance Pitch (Mostaqel / Gulf Market):

"I built Sakina — a production-ready bilingual Arabic/English mental health AI chatbot with real-time emotion detection, a cartoon avatar that reacts to the user's emotional state, hybrid RAG retrieval, and a crisis safety protocol. Built with Lightning AI, FastAPI, Qdrant, and React. I can adapt this for: HR wellness bots, empathetic customer support, educational chatbots that adapt to student frustration, and Arabic-language mental health clinics."

Final GitHub README Checklist:

- Architecture diagram (Excalidraw or Mermaid) — all 5 modules + UI + data flow
- 2-minute screen-recording demo GIF: all 6 emotions → avatar reacts → crisis overlay
- Results table: all modules, all metrics, in one clean comparison table
- Quick Start: docker compose up → full system in 3 commands
- HuggingFace model links for emotion classifier with model card
- Research papers cited — shows academic grounding, impresses senior engineers
- Arabic description section (■■■■ ■■■■ ■■■■■■) — extremely rare, very impressive to Gulf clients
- Ethics & Limitations section — shows maturity and responsible AI thinking
- Live demo link if deployed (Lightning AI free tier or HuggingFace Spaces)

Sakina is not a project — it is a product statement.

"I build AI that feels human, ships in production, and earns trust."

Omar Gamal ElKady · ITI AI Track, Intake 46 · NLP Final Project 2026